Week – 11 Practical

1.Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

vedadripinninti@MSI123

GNU nano 8.7.1 src/race_condition.c *

```c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        counter++;
    }

    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter: %d\n",
            NUM_THREADS * ITERATIONS);

    printf("Actual counter: %d\n", counter);

    return 0;
}
```

Output:

vedadripinninti@MSI123

```
vedadripinninti@MSI123:~$ cd OS_Skill
mkdir -p Week-11/src Week-11/bin
cd Week-11
vedadripinninti@MSI123:~/OS_Skill/Week-11$ nano src/race_condition.c

vedadripinninti@MSI123:~/OS_Skill/Week-11$ gcc src/race_condition.c -o bin/race_condition -pthread
vedadripinninti@MSI123:~/OS_Skill/Week-11$ ./bin/race_condition
Expected counter: 400000
Actual counter: 164373
vedadripinninti@MSI123:~/OS_Skill/Week-11$
```

2. Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

vedadripinninti@MSI123

GNU nano 8.7.1 src/mutex_counter.c *

```c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

pthread_mutex_t lock;

void *increment(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {

        pthread_mutex_lock(&lock);

        counter++;
```

```c
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter: %d\n",
           NUM_THREADS * ITERATIONS);

    printf("Actual counter: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output:

```
vedadripinninti@OSI:~/OSI_Skill/Week-11$ nano src/mutex_counter.c

vedadripinninti@OSI:~/OSI_Skill/Week-11$ gcc src/mutex_counter.c -o bin/mutex_counter -pthread
vedadripinninti@OSI:~/OSI_Skill/Week-11$ ./bin/mutex_counter
Expected counter: 400000
Actual counter: 400000
vedadripinninti@OSI:~/OSI_Skill/Week-11$ ./bin/mutex_counter
Expected counter: 400000
```